

Manual Tecnico - Graficos Divertidos

Manual técnico — Gráficos Divertidos

1. Stack y versiones

Componente	Versión
Kotlin	2.0.21
Android Gradle Plugin (AGP)	8.5.2
Gradle Wrapper	8.7
KSP	2.0.21-1.0.28
Jetpack Compose BOM	2024.09.00
Material 3	(vía BOM)
Navigation Compose	2.8.0
Room	2.6.1
Coroutines	1.8.1
JDK	17
minSdk / compileSdk / targetSdk	24 / 34 / 34

Todas las versiones están fijadas explícitamente en `gradle/libs.versions.toml` (catálogo de versiones); no se usan rangos dinámicos (`+`) ni `latest` .

2. Arquitectura

MVVM + Repository Pattern:

```
ui/screens/* -> ViewModel (StateFlow) -> Repository ->
DAO (Room) -> SQLite
\ domain/logic (Kotlin puro)
```

- **domain/model** : modelos inmutables sin dependencias de Android (DatasetModel, ExerciseModel, etc.).
- **domain/logic** : ChartMathEngine (matemática de gráficos) y GamificationEngine (niveles, rachas, insignias) — Kotlin puro, testeable con JUnit sin Robolectric.
- **data/local** : entidades Room, DAOs, Converters , AppDatabase , y el paquete seed (contenido semilla + DatabaseSeeder).
- **data/repository** : ProfileRepository , ContentRepository , ProgressRepository , BuilderRepository . ProgressRepository es el único punto de escritura para intentos/progreso/XP/insignias, y usa AppDatabase.withTransaction para que cada intento se registre de forma atómica.
- **ui** : Compose. theme/ (colores, tipografía, formas), components/ (Canvas de gráficos, widgets comunes, mascota Grafi), navigation/ (rutas + fábricas de ViewModel sin Hilt), screens/ (una carpeta por módulo).

No se usa Hilt/Dagger: la inyección es manual desde GraficosDivertidosApp (contenedor de dependencias por lazy), suficiente para el tamaño del proyecto y más simple de

auditar/compilar.

3. Estructura de carpetas

```
app/src/main/kotlin/com/educalab/graficosdivertidos/
|-- GraficosDivertidosApp.kt      Contenedor de dependencias
|-- MainActivity.kt               Punto de entrada, seeding,
onboarding gate
|-- data/
|   |-- local/
|   |   |-- entity/              14 entidades Room (ver
BASE_DE_DATOS.md)
|   |   |-- dao/                 7 interfaces DAO
|   |   |-- converters/          Converters.kt (listas ->
texto delimitado)
|   |   |-- seed/                SeedModels, SeedContent
(generator), DatabaseSeeder
|   |   |-- AppDatabase.kt
|   |-- repository/              4 repositorios + Mappers
|-- domain/
|   |-- model/                   ChartModels,
CompositeModels
|   |-- logic/                   ChartMathEngine,
GamificationEngine
|-- ui/
|   |-- theme/, components/, navigation/, AppState.kt
|   |-- screens/
|       |-- onboarding/, home/, module/, builder/
|       |-- comparator/, detective/, gallery/, profile/
```

4. ViewModels

Cada pantalla tiene un ViewModel con `StateFlow` de un `data class` `UiState` inmutable. Ejemplos: `ModuleExerciseViewModel` conduce tanto las sesiones por módulo como el modo “repaso” del módulo Desafíos (cuando `moduleKey == null`, delega en `ContentRepository.getPendingReviewExercises`). `BuilderViewModel` modela el wizard de 7 pasos con un enum `BuilderStep` y valida con `ChartMathEngine.validateChartConfiguration` antes de habilitar “Guardar”.

Las fábricas de ViewModel (`ui/navigation/ViewModelFactories.kt`) usan la DSL `viewModelFactory { initializer { ... } }` de `androidx.lifecycle.viewmodel`, inyectando los repositorios desde `GraficosDivertidosApp` sin generación de código adicional.

5. Room: entidades, DAOs y transacciones

Ver `docs/BASE_DE_DATOS.md` para el modelo completo. Puntos técnicos relevantes:

- Las listas de primitivos (categorías seleccionadas, opciones de respuesta, respuesta seleccionada) se guardan como texto delimitado (`StringListCodec` / `IntListCodec`) en lugar de JSON, para no depender de `org.json` (que en pruebas JVM sin Robolectric es solo un stub que lanza excepción) y mantener la lógica 100% testeable con JUnit puro.
- `ProgressRepository.recordExerciseAttempt/recordErrorChallengeAttempt/recordComparisonAttempt` envuelven cada registro en `db.withTransaction { ... }`: se inserta el intento, se recalcula el progreso del módulo, se recalculan XP/racha/estrellas y se evalúan insignias nuevas, todo o nada.
- `@Transaction` se usa también en `DatasetDao.insertDatasetWithPoints` para insertar un dataset y sus puntos de forma atómica durante el seeding.
- Las claves foráneas usan `onDelete = CASCADE` de forma consistente (borrar un dataset borra sus puntos, definiciones de gráfico, retos del detective y del comparador asociados).

6. Lógica de dominio testeable

`ChartMathEngine` (escalado de ejes con “nice numbers”, normalización a porcentajes con corrección de residuo de redondeo, cálculo de sectores circulares, conteo de iconos de pictograma, layout de barras, coordenadas de líneas, validación de configuración del Constructor) y `GamificationEngine` (niveles por XP, rachas, estado de módulo, reglas de insignias) no importan nada de `android.*` ni de Compose, por lo que se prueban con JUnit + Truth sin necesidad de Robolectric.

7. Dependencias principales

Ver `app/build.gradle.kts` y `gradle/libs.versions.toml`. Resumen: Compose (ui, material3, material-icons-extended, navigation-compose), Room (runtime + ktx + compiler vía KSP), Coroutines (core + android + test), y para pruebas: JUnit4, Truth, Robolectric, `androidx.room:room-testing`, `androidx.arch.core:core-testing`.

8. Permisos

Ninguno. `AndroidManifest.xml` no declara `INTERNET` ni ningún otro `<uses-permission>`.

9. Build

```
./gradlew clean
./gradlew assembleDebug
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebugBundle
```

`.github/workflows/build.yml` reproduce estos mismos pasos en GitHub Actions (Ubuntu, JDK 17, Android SDK vía `android-actions/setup-android`) y publica el APK debug, el reporte de tests y el reporte de lint como artefactos al hacer push.

10. Pruebas

73 pruebas JVM (`app/src/test/kotlin`):

- `domain/ChartMathEngineTest.kt` (30 tests): nice numbers, porcentajes, sectores circulares, pictogramas, layout de barras, coordenadas de líneas, validación de configuración, casos límite (listas vacías, división por cero, valores negativos).
- `domain/GamificationEngineTest.kt` (20 tests): niveles, XP, estados de módulo, rachas, precisión, reglas de insignias, desbloqueo de módulos avanzados.
- `data/ConvertersTest.kt` (7 tests): codificación/decodificación reversible de listas y enums, texto con acentos.
- `data/DatabaseSeederTest.kt` (8 tests, Robolectric + Room en memoria): conteos exactos del contenido semilla, idempotencia, cascada de borrado.
- `data/ProgressRepositoryTest.kt` (8 tests, Robolectric + Room en memoria): XP otorgado, desbloqueo de insignias, no duplicación de insignias, actualización de progreso, reinicio de racha, primer intento vs. reintento, progreso del Constructor.

Además, 1 prueba instrumentada (`app/src/androidTest/kotlin/.../MainActivitySmokeTest.kt`) que arranca `MainActivity` en un dispositivo/emulador real y verifica que la app siembra la base de datos y llega al onboarding o a la Home sin bloquearse.

11. Mantenimiento y ampliación

- **Agregar contenido semilla:** editar las listas en `tools/generate_seed_content.py` y volver a ejecutar `python3`

`tools/generate_seed_content.py` ; regenera `SeedContent.kt` completo.

- **Agregar un módulo nuevo:** añadir el valor al enum `ModuleKey` , su título en `moduleTitles / moduleDescriptions` , su icono en `IconMapping.kt` , y su pantalla en `ui/screens/` + ruta en `NavGraph.kt` .
- **Agregar un tipo de gráfico:** añadir el valor a `ChartType` , su rama en `ChartRenderer` y su Canvas en `ChartCanvases.kt` ; toda la matemática de escalado ya es reutilizable desde `ChartMathEngine` .
- **Regenerar las ilustraciones:** `python3 tools/generate_art.py` reescribe todos los PNG de `res/drawable-nodpi/` .